Deep Dive

LLM APIs & Applications

Overview of Today's Topics

Key Learning Objectives

- Understanding Chat Completions API
- Types of LLMs: Base, Chat, Reasoning
- The Transformer Architecture Explained
- Building a Multi-Step Application

The Chat Completions API

WHAT IS IT?

The Chat Completions API represents the **standard way** to interact with large language models (LLMs). It was invented by OpenAI and has been widely adopted across various platforms.

THE CORE IDEA

This API functions by allowing users to input a conversation and, based on that input, it predicts and generates the **most likely response** in a natural conversational flow.

NAMED BECAUSE

The name stems from its functionality: you provide chat history and receive a **completion** in return, facilitating seamless interactions with AI-driven conversational agents.

Behind the Scenes

WHAT REALLY HAPPENS

Your code produces an HTTP request which is sent to the OpenAI server, where the request is processed, and a response is generated based on the provided input.

THE ENDPOINT

The specific endpoint to access the Chat Completions API is: POST https://api.openai.com/v1/chat/completions. This is where your requests are directed for processing.

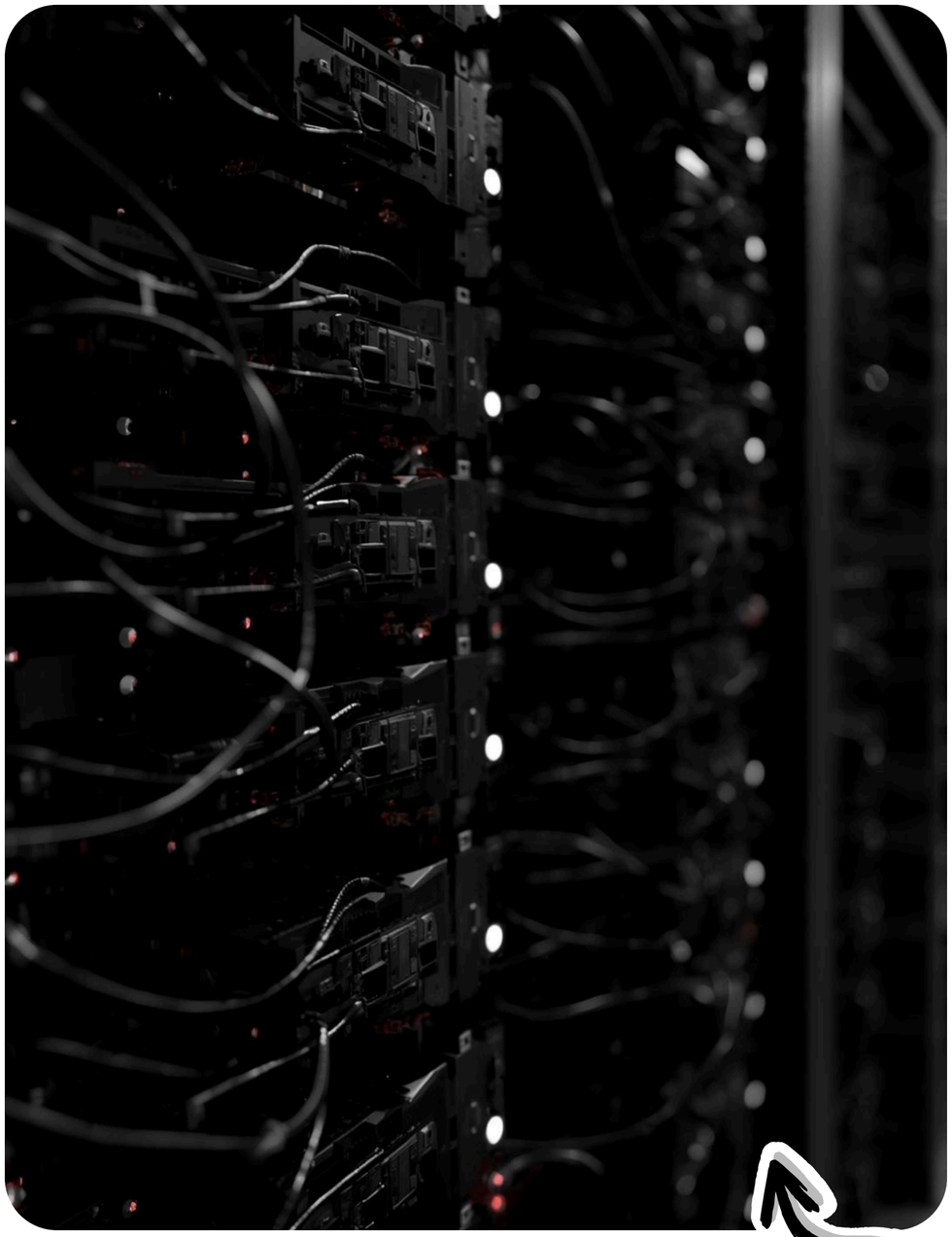
The OpenAI Python Library Overview

What It Actually Is

- Just a lightweight wrapper around HTTP calls
- Makes API calls easier to write
- Not running any AI locally!

Why Use It?

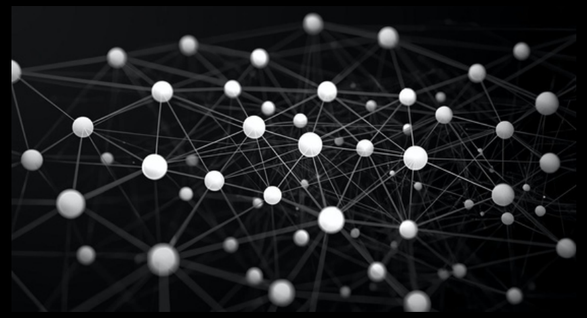
- Cleaner code
- No manual JSON handling
- Built-in error handling



One Library, Multiple Providers

OpenAI's Python library standardizes API interactions, enabling seamless transitions between different providers without changing code, fostering flexibility and efficiency in development processes.

Three Types of LLMs Explained

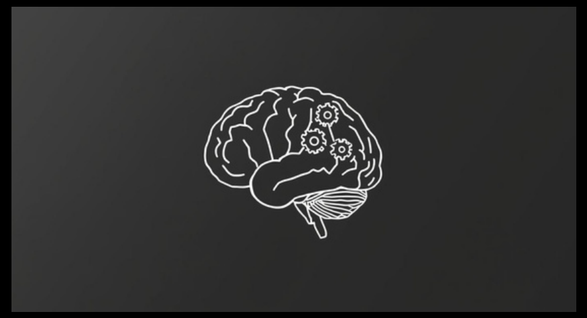


Base Model

Predicts the next token only, like autocomplete.

Chat Model

Trained for conversations, enabling interactive dialogues.



Reasoning Model

Thinks before answering, enhancing response accuracy.

Base Models

Understanding their core functions

Definition

Base models **only predict what follows** the input text. They lack conversational abilities and function similarly to advanced autocomplete systems, focusing solely on text completion.

Example

For instance, given the input "The capital of France is," a base model outputs "Paris, which is known for its rich culture and history," demonstrating its predictive capability.

When to Use

Base models are ideal for **fine-tuning** tasks and developing specialized models. They are particularly useful when tailored predictions are needed for specific applications or industries.

Chat Models

Definition

Trained to follow instructions effectively.

Training Method

Human ratings improve responses through RLHF.

Characteristics

Fast responses suitable for creative content.

Reasoning Models

Definition

Thinks step-by-step before answering questions.

Wait Trick

Adding "Wait" helps reconsider the answer.

Use Cases

Ideal for math problems and logic puzzles.

Hybrid Models

REASONING CAPABILITY

Hybrid models, such as GPT-5 and Gemini 2.5 Pro Models can engage in reasoning when complex decision-making is required for tasks.

ADAPTIVE THINKING

Models autonomously determine the extent of reasoning needed based on the context.

ENHANCED USER EXPERIENCE

Users benefit from seamless interactions that adapt to their specific needs and demands.

FAST INTERACTION

Quick chat responses are available when in-depth reasoning is not necessary.

VERSATILE APPLICATIONS

Hybrid models are well-suited for diverse applications, balancing speed and accuracy.

The Transformer Story

2017

Google researchers publish a groundbreaking paper introducing a new neural network architecture called the Transformer.

ARCHITECTURE

This innovative architecture fundamentally changes how natural language processing tasks are approached through effective use of attention mechanisms.

KEY INNOVATION

The attention mechanism allows for more efficient processing of sequences, improving performance across various language tasks.

Why the Transformer Model Works

EFFICIENT PROCESSING

The Transformer architecture processes sequences **efficiently**, allowing it to handle long-range dependencies in data without losing contextual relevance or requiring excessive computational resources.

PARALLEL EXECUTION

Unlike LSTM models, Transformers can **run in parallel**, significantly speeding up training and inference times by allowing simultaneous processing of multiple data points across different layers.

SCALABILITY

This architecture **scales effectively** with increasing data and computational power, making it capable of handling larger datasets and more complex models without compromising performance or accuracy.

The Rise of GPT

GPT-1

Launched in 2018, GPT-1 laid the foundation for transformer-based models, showcasing the potential of generative pre-training with 117 million parameters to process natural language.

GPT-2

Released in 2019, GPT-2 significantly increased model complexity with 1.5 billion parameters, demonstrating the ability to generate coherent and contextually relevant text across diverse topics.

GPT-3

In 2020, GPT-3 further advanced AI capabilities with 175 billion parameters, providing enhanced understanding of context and nuance, leading to a wide range of applications in natural language processing.

GPT-4

Introduced in 2023, GPT-4 boasted a remarkable 1.76 trillion parameters, pushing the boundaries of AI performance and efficiency, marking a new era in the evolution of language models.

Understanding Parameters

The building blocks of AI models

Simple Definition

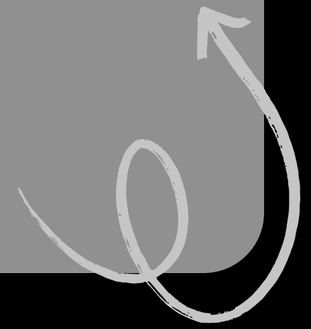
The parameters represent the "knowledge" stored within the model, allowing it to understand and generate text based on patterns learned from vast amounts of data.

Think of It Like

More parameters can be likened to having more "brain cells," enabling the model to learn and recognize intricate patterns, resulting in enhanced performance and capabilities.

Key Insight

Recent advancements have improved efficiency, allowing smaller models to achieve greater tasks, proving that more parameters are not always necessary for effective learning and performance.



Training Time vs Inference

Scaling Approaches Explained

Training Time Scaling

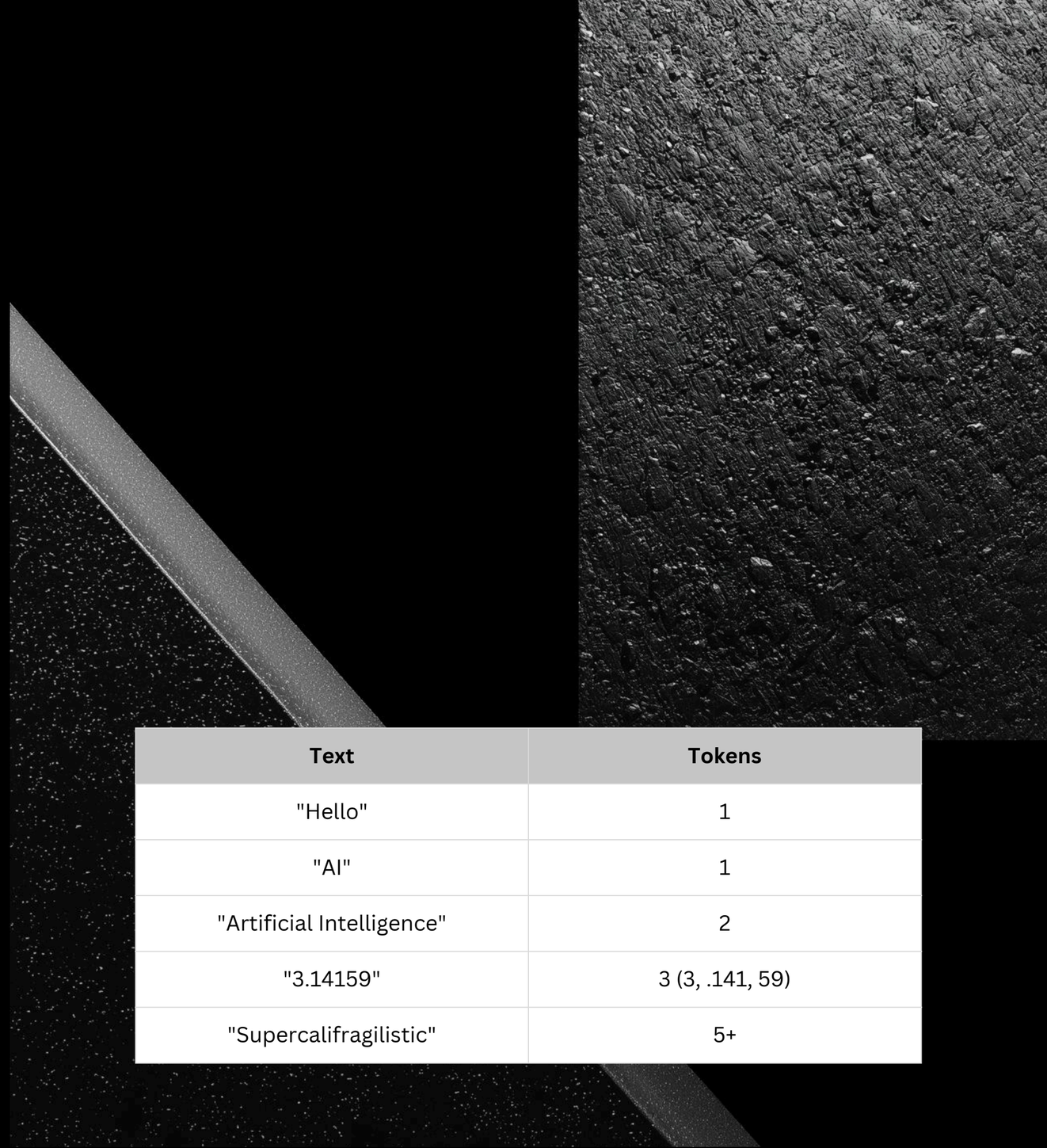
A larger model requires **more parameters**, which leads to increased complexity and demands **more training data**. While this approach has an **expensive upfront cost**, it can significantly enhance the model's capabilities and overall performance.

Inference Time Scaling

Leveraging the same model, this approach focuses on utilizing **better prompts** (RAG) and implementing **more reasoning** with thinking tokens. While it incurs an **expensive per-use cost**, it can effectively improve the results of the model's output.

Understanding Tokens in AI

- Chunks of text processed by the model efficiently
- Not characters or words, but segmented units for precision
- Rule of thumb: approximately four characters per token
- ~1 token ≈ 4 characters ≈ 0.75 words



Text	Tokens
"Hello"	1
"AI"	1
"Artificial Intelligence"	2
"3.14159"	3 (3, .141, 59)
"Supercalifragilistic"	5+

Tokenization Insights

Understanding Its Importance in AI

COST IMPLICATIONS

Tokenization directly influences operational costs, making it crucial to understand for budgeting and resource allocation.

CONTEXT LIMITATIONS

The number of tokens affects the context limits, which can impact how language models interpret input data.

TEXT PERCEPTION

Tokenization plays a vital role in how models "see" text, shaping their understanding and response to queries.

The Illusion of Memory

Critical Understanding

Every API call is stateless! This fundamental characteristic influences how the model operates, limiting its ability to track ongoing interactions and contextual nuances across requests.

What This Means

The model does not retain memory of previous calls. Each interaction stands alone, requiring the user to provide all necessary context with each new request for clarity.

Creating "Memory"

To simulate memory, it is essential to pass the entire conversation history with every API call. This allows the model to process context and respond appropriately.

Importance of Context

Providing complete context in each request is vital. It ensures that the model generates relevant and coherent responses, enhancing the overall user experience and interaction quality.

Memory in Practice: Understanding Calls

First Call

User introduces themselves to the system.

```
messages = [
    {"role": "user", "content": "My
name is John"}
]
```

Response: "Nice to meet you, John!"

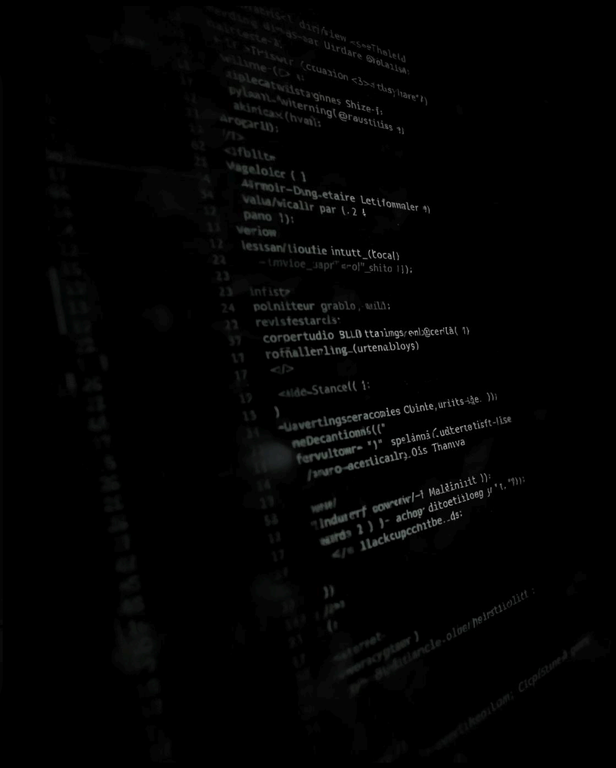
Second Call

User asks a question without context.

```
messages = [
  {"role": "user", "content": "What's
my name?"}
]
```

Response: "I don't know your name"

Creating the Illusion of Memory



```
messages = [
    {"role": "user", "content": "My
name is John"},
    {"role": "assistant", "content":
"Nice to meet you!"},
    {"role": "user", "content": "What's
my name?"}
]
```

Response: "Your name is John"

The Trick

Include previous messages for context.

ChatGPT

Remembers user inputs to provide accurate responses.

Context Window

Understanding Token Processing Limits

The context window defines the maximum tokens a model can process, including the system prompt, conversation history, new messages, and the generated response. It’s crucial for performance.

Model	Context Window
GPT-5	400,000 tokens
Claude	200,000 tokens
Gemini	1,000,000 tokens

API Costs

Understanding Pricing Structure for Tokens

Pricing Structure

Input tokens are cheaper, while output tokens carry a higher cost, impacting overall usage expenses for users.

Example: GPT-5

For GPT-5, input tokens are priced at $1.25 per million, whereas output tokens are significantly higher at $10 per million.

GPT-5 Nano

The GPT-5 Nano option offers a budget-friendly pricing, with input tokens costing $0.05 and output tokens at $0.40 per million.

Understanding Costs

01

Perspective

1 million tokens ≈ Complete Works of Shakespeare

02

Simple Chat

Simple chat costs are fractions of a cent

03

Hidden Costs

Reasoning tokens represent a hidden cost

04

Cost Traps

Long conversation histories can incur high costs

05

Agent Loops

Agents/loops can add up quickly in costs

One-Shot Prompting

One-shot prompting allows users to provide a single example of desired output, which enables the model to learn the pattern effectively. This approach streamlines communication and enhances response accuracy.

```
"Respond in this JSON format:
{
  "type": "about page",
  "url": "https://example.com/about"
}"
```

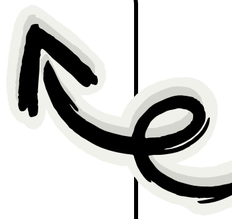
JSON Response Format

Forcing JSON Output

This approach **ensures the model** outputs valid JSON only, preventing errors and enhancing data integrity during inference, which is crucial for effective programming.

```
response = client.chat.completions.create(
 model="gpt-4o-mini",
 messages=messages,
 response_format={"type": "json_object"}
)
```

Why It Works



By constraining the model, it consistently generates outputs in the required format, facilitating easier processing and integration with existing systems, thereby improving overall efficiency.

Streaming Responses Explained

What Is Streaming?

Streaming allows you to receive tokens as they are generated, creating a **typewriter effect** that enhances the interaction experience with the model.

How to Enable

To enable streaming, set the stream parameter to True when calling the API.

```
response =
client.chat.completions.create(
  model="gpt-4o-mini",
  messages=messages,
  stream=True
)
```

Benefits

Streaming responses provide a **better user experience** by allowing immediate progress visibility and the option to cancel the process early if necessary.

Building Multi-Step Applications

01

First LLM Call

Analyze and extract initial data

02

Process Results

Refine and prepare input for next step

03

Second LLM Call

Generate and transform final output

Brochure Generator

01

Get Website Links

Retrieve all necessary website URLs.

02

Select Relevant Links

LLM identifies links in JSON format.

03

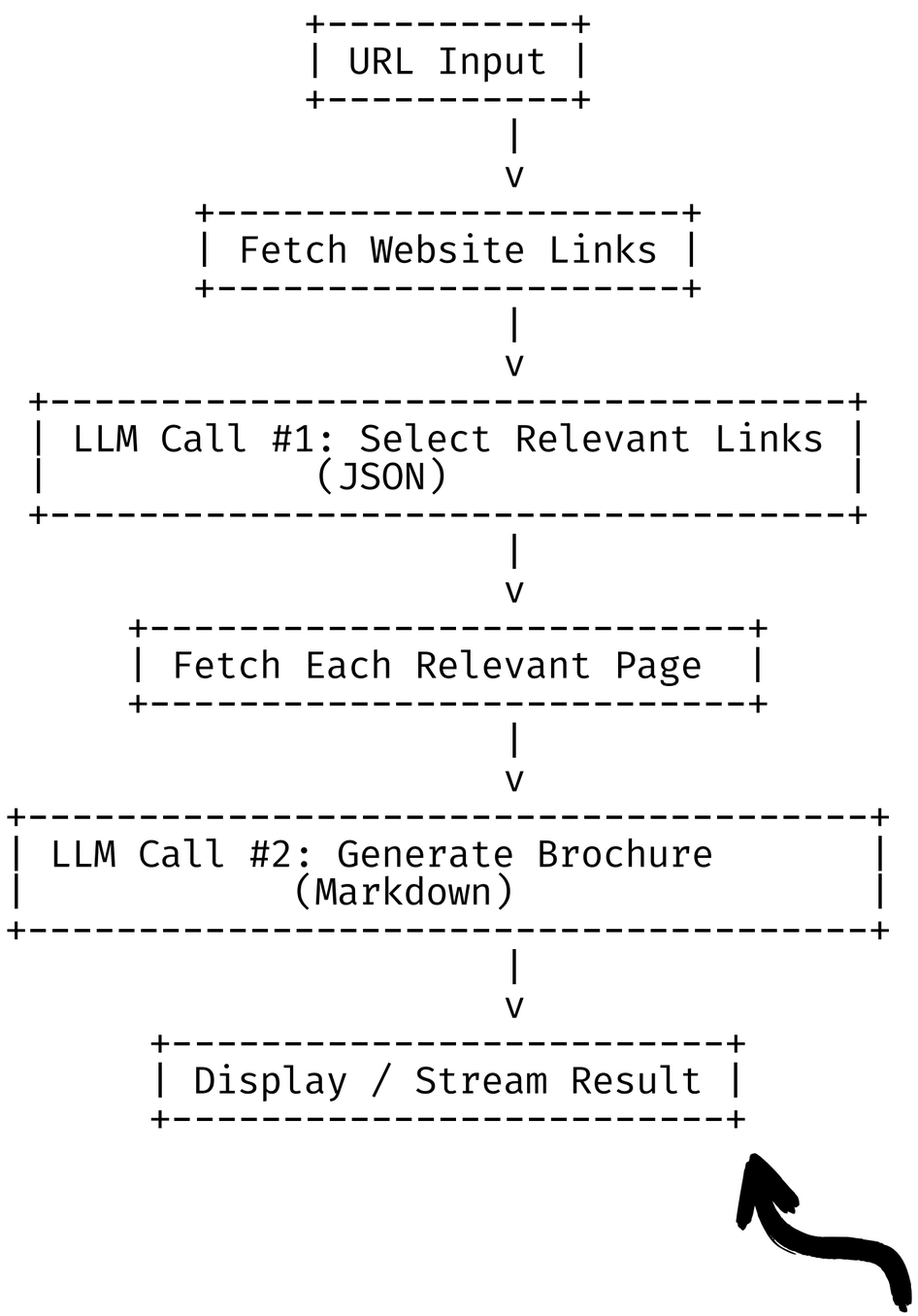
Fetch Linked Pages

Gather content from selected URLs.

04

Generate Brochure

Brochure Output in Markdown



Practical Application Flow

This diagram illustrates the **step-by-step** process of building multi-step applications, capturing the essential flow from input to final results efficiently and clearly.

LLM Development: 5 Essential Points

01

API Calls

API calls are HTTP requests that simplify interactions.

02

Memory Illusion

Memory is an illusion; pass full history every time.

03

Tokens Impact

Tokens influence costs, limits, and understanding significantly.

04

Multiple Calls

Chain multiple LLM calls for complex tasks.

05

Experiment Constantly

Iterate on prompts for improved results and efficiency.

Digital You — Personalized Digital Twin with LLMs

🔗 **GitHub Repository:**
https://github.com/smqd19/Digital-You.git

This repository serves as the foundation for building a personalized Digital Twin using Large Language Models (LLMs).

Project Goal

The goal of this project is to ingest professional artifacts —such as:

- Resumes
- LinkedIn profiles
- Portfolios

…and expose them through a conversational AI interface that accurately represents an individual's professional persona.

Enhance your skills with practical tasks

Here are some exercises to solidify your understanding and skills:

1. Build a **multi-step application** to see how different components interact.
2. Experiment with **JSON outputs** to enhance data handling.
3. Try **streaming responses** for real-time interactions.
4. Compare **costs across models** to understand pricing strategies and efficiency.

Engaging in these activities will foster a deeper comprehension of LLM applications and their practical implications in real-world scenarios.

Thank You